Handwritten Mathematical Expression Recognition with a Classic Attention-LSTM: Staged Transfer Learning and a Comparison with a Transformer Baseline

Hardik Khaneja
Manipal Institute of Technology
Manipal, India

Dhruva R M Upadhyaya
Manipal Institute of Technology
Manipal, India

Rijul Bhatia
Manipal Institute of Technology
Manipal, India

***Abstract*—We study image-to-LaTeX recognition of mathematical expressions with a deliberately classic encoder–decoder: a ResNet-18 convolutional encoder, additive (Bahdanau) attention and an LSTM decoder with beam search. The model is trained in three stages that move progressively further from clean, printed input: printed formulas (im2latex-100k), handwritten expressions (MathWriting) and photographed handwriting (HME100K). A single 429-token vocabulary is shared by all stages, so each stage warm-starts from the previous one without resizing the output layer. Images are resized with their aspect ratio preserved and attention is masked so that padding is never attended to. On held-out subsets the model reaches a BLEU of 91.5 on printed formulas and 79.4 on handwritten expressions, with expression-level exact match (ExpRate) of 0.47 and 0.32 respectively. We compare against pix2tex, an open-source vision-transformer model, under an identical tokenizer and metric implementation; used off the shelf, it recognizes none of 500 handwritten test expressions exactly, showing the size of the printed-to-handwritten domain gap; on printed formulas it reaches BLEU 87.3 against our 91.5, and on photographed handwriting it scores BLEU 4.0, and ablate a finer encoder grid, coverage attention and photo-style augmentation: in a 10-epoch, single-seed fine-tune, all four variants lie within 2 BLEU points and 2.4 ExpRate points of each other, inside the sampling error of a 500-image test subset.**

***Keywords*—handwritten mathematical expression recognition, image-to-markup, attention, encoder–decoder, transfer learning**

I. Introduction

Converting an image of a mathematical expression into LaTeX is harder than ordinary text recognition. Symbols are arranged in two dimensions (fractions, sub- and superscripts, radicals), the output is a long structured token sequence, and handwriting adds large variation in stroke shape, size and spacing. Photographs of handwriting add a further domain gap: perspective, uneven lighting, blur and compression artefacts.

Recent systems for this task use transformer decoders and, increasingly, vision-transformer encoders. Such models are data- and compute-hungry. This paper asks a practical question: *how far does a classic, interpretable CNN–attention–LSTM model go when trained on a limited compute budget, and what does a modern transformer model buy over it?*

Our contributions are:

- A from-scratch implementation of a ResNet-18 + Bahdanau attention + LSTM image-to-LaTeX model with aspect-preserving preprocessing and pad-masked attention (Section III).
- A three-stage transfer-learning recipe (printed → handwritten → photographed) with a single shared vocabulary, and measurements after each stage (Section V).
- A comparison with pix2tex using one shared tokenizer and metric implementation, so both models are scored identically.
- An ablation of three design choices: encoder grid resolution, coverage attention and photo-style data augmentation.

We frame the study around three research questions:

RQ1 How does the classic model compare with a transformer-based model (pix2tex) on the same test images?

RQ2 How does accuracy change across the printed, handwritten and photographed stages?

RQ3 How much does each of the finer encoder grid, coverage attention and photo-style augmentation contribute?

II. Related Work

Image-to-markup. Deng et al. [1] framed formula recognition as image-to-markup generation with a convolutional encoder and attention-based recurrent decoder, and released the im2latex-100k dataset of rendered formulas. Zhang et al. [2] proposed Watch, Attend and Parse (WAP) for handwritten expressions, using a fully convolutional encoder and an attention-based GRU decoder with a coverage vector.

Attention. Additive attention was introduced by Bahdanau et al. [3] for machine translation. Xu et al. [4] applied it to image captioning and introduced a doubly stochastic regularizer that encourages every image region to receive attention

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

over the course of decoding. Tu et al. [5] introduced coverage, which feeds accumulated past attention back into the attention model to reduce repeated and skipped inputs.

Transformer-based recognizers. pix2tex (LaTeX-OCR) [7] is an open-source model that pairs a vision-transformer encoder with a transformer decoder, trained primarily on rendered formulas.

Datasets. MathWriting [8] is a large dataset of online handwritten expressions, of which we use the human-written portion in pre-rendered image form. HME100K [9] consists of photographed handwritten expressions captured in real-world conditions.

III. Methodology

A. Preprocessing

Each image is converted to grayscale and replicated to three channels so that the ImageNet-pretrained first convolution remains valid. It is then scaled to a fixed height of 128 pixels *preserving its aspect ratio*; only images wider than 1024 pixels after scaling are scaled down further, still without distortion. Images in a batch are right-padded to the widest image, and each sample carries its valid height and width so that the padding can be masked out of attention. Pixel values are normalized with ImageNet statistics.

B. Encoder

The encoder is a ResNet-18 [6] with its average-pool and classification layers removed, followed by a 1×1 convolution projecting 512 channels to $d_e = 256$. The $H' \times W'$ feature map is flattened into a sequence of $H'W'$ annotation vectors $\{\mathbf{e}_i\}$. By default the backbone has total stride 32; the *fine-grid* variant sets the stride of the first block of the last stage to 1, giving total stride 16 (8 feature rows rather than 4 at height 128).

C. Attention

At decoding step t, with decoder hidden state $\mathbf{h}_{t-1}$, attention energies are

$$s_{t,i} = \mathbf{v}^\top \tanh\big(W_e \mathbf{e}_i + W_d \mathbf{h}_{t-1} + \mathbf{w}_c \, c_{t,i}\big), \tag{1}$$

where $c_{t,i} = \sum_{k<t} \alpha_{k,i}$ is the coverage of cell i. The coverage term is present only in the coverage variant; its weights are initialized to zero, so a model warm-started from a checkpoint without coverage initially produces identical outputs. Energies at padded cells are set to a large negative value before the softmax, so $\alpha_{t,i} \approx 0$ on padding. The context vector is $\mathbf{z}_t = \sum_i \alpha_{t,i} \mathbf{e}_i$.

D. Decoder

An LSTM cell with hidden size 256 takes the concatenation of a 128-dimensional token embedding and $\mathbf{z}_t$. Its initial hidden and cell states are linear projections of the mean of the valid (unpadded) annotation vectors. Output logits are a linear function of $[\mathbf{h}_t; \mathbf{z}_t]$ after dropout (0.3). The attention dimension is 128.

TABLE I
Data used at each training stage.

Stage	Dataset	Train	Val	Test
1	im2latex-100k	55,033	6,072	6,810
2	MathWriting (human)	40,000	4,000	7,644
3	HME100K (self-split)	≈70k	≈2k	≈2k

E. Training objective

The loss is token-level cross-entropy (padding ignored) plus the doubly stochastic attention regularizer of [4]:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \cdot \tfrac{1}{N} \sum_i \Big(1 - \textstyle\sum_t \alpha_{t,i}\Big)^2, \qquad \lambda = 1. \tag{2}$$

Training uses teacher forcing with ratio starting at 1.0, reduced by 0.02 per epoch to a floor of 0.5.

F. Inference and confidence

We decode with beam search (width 5, maximum length 200) and rank finished hypotheses by $\log p / L^{0.7}$, where L is the hypothesis length. The reported confidence is the length-normalized probability $\exp(\log p / L)$ of the selected hypothesis.

G. Tokenization

LaTeX strings are split by a regular-expression tokenizer into commands (e.g. `\frac`), single characters and braces. A vocabulary of 429 tokens (including four special tokens: padding, start, end and unknown) is built once from the im2latex-100k training set and reused unchanged for every later stage.

IV. Experimental Setup

A. Datasets

Table I summarizes the data used at each stage.

- **Stage 1 (printed):** im2latex-100k [1], using its train, validation and test splits.
- **Stage 2 (handwritten):** the human-written portion of MathWriting [8], capped at 40,000 training and 4,000 validation images. The synthetic portion is not used. The stage-1 vocabulary covers 99.9% of MathWriting tokens.
- **Stage 3 (photographed):** HME100K [9]. The public mirror we used provides labels for the training set only, so we split it into training, validation and test portions of approximately 70k, 2k and 2k images. *Our HME100K test set is therefore a held-out slice of the original training distribution, not the official HME100K test set, and our numbers are not comparable with published HME100K results.*

B. Training

Stage 1 trains from ImageNet-initialized ResNet-18 weights with Adam (learning rate 10^{-3}, weight decay 10^{-5}), batch size 32 and gradient-norm clipping at 5.0. The learning rate is reduced on a validation-loss plateau (patience 2). Stages 2 and 3 warm-start from the previous stage's best checkpoint

TABLE II
BASELINE MODEL AFTER EACH TRAINING STAGE (TEST SUBSETS OF 500 IMAGES; HME100K ON OUR SELF-SPLIT HELD-OUT SLICE).

Stage	Test set	BLEU	ExpRate	≤1	≤2
1	im2latex (printed)	91.5	0.470	0.550	0.700
2	MathWriting (hand.)	79.4	0.320	0.510	0.650
3	HME100K (photos)	88.3	0.524	0.724	0.832

with learning rate 10^{-4}. The best checkpoint at each stage is selected by validation cross-entropy. The best checkpoint was reached at epoch 32 in stage 1, epoch 25 in stage 2 and epoch 14 in stage 3, out of 32, 30 and 15 epochs run respectively. All training was run on Kaggle (single NVIDIA Tesla T4, 16 GB); wall-clock time was approximately 3.5, 2.5 and 2 hours for the three stages.

Training-time augmentation (applied on the fly): random affine transform (rotation and shear up to 5°, translation up to 2%), Gaussian blur and brightness/contrast jitter. The *photo-style* augmentation used in the ablation adds random perspective distortion, a linear shadow gradient, stronger blur and jitter, and JPEG re-encoding at quality 20–60.

C. Model variants

All results in Tables II–III use the *baseline* configuration: total encoder stride 32, no coverage, and the lighter augmentation above. The ablation (Table IV) adds the fine grid, coverage and photo-style augmentation one at a time.

D. Metrics

Predictions and references are tokenized with the same tokenizer and scored by one shared implementation for every model:

- **BLEU**: corpus-level 4-gram BLEU [10] over tokens (NLTK, smoothing method 4).
- **ExpRate**: fraction of expressions whose predicted token sequence matches the reference exactly.
- **ExpRate** $\leq k$: fraction whose token-level edit distance to the reference is at most k, for $k \in \{1, 2\}$.

V. RESULTS AND ANALYSIS

A. RQ2: Accuracy across stages

Table II reports each stage's best checkpoint on its own test set. These figures are computed on the first 500 images of each test split, and on the self-split HME100K test slice.

Moving from printed to handwritten input lowers BLEU from 91.5 to 79.4 and ExpRate from 0.47 to 0.32, reflecting the added difficulty of handwriting. Fine-tuning on MathWriting raised validation token accuracy from 0.52 (the stage-1 model evaluated on MathWriting) to 0.82. Stage-3 validation token accuracy peaked at 0.836. The higher ExpRate on HME100K than on MathWriting should not be read as photographs being easier: the HME100K test slice comes from the same distribution as its training data, and the two test sets differ in expression length and symbol distribution. The 500 MathWriting test expressions average 18.1 tokens (median 17, range 1–72) The 2,000 HME100K test expressions average 17.1 tokens (median 14, range 1–90), so they are not longer than the MathWriting ones; the gap is better explained by the in-distribution test slice and by HME100K's narrower range of expression types..

TABLE III
CLASSIC MODEL VS. PIX2TEX ON IDENTICAL TEST IMAGES. †SEE TEXT: THE PIX2TEX HME100K ROW USES A RE-BUILT 2,000-IMAGE SLICE THAT MAY DIFFER FROM THE SLICE BEHIND OUR HME100K ROW.

Model	Test set	BLEU	ExpRate	≤1	≤2
Ours	im2latex	91.5	0.470	0.550	0.700
pix2tex	im2latex	87.3	0.352	0.422	0.524
Ours	MathWriting	79.4	0.320	0.510	0.650
Ours, full MW	MathWriting	86.7	0.478	0.680	0.794
pix2tex	MathWriting	0.7	0.000	0.000	0.000
Ours	HME100K	88.3	0.524	0.724	0.832
pix2tex	HME100K†	4.0	0.000	0.0005	0.003

More handwriting data. Training on the full MathWriting human split for 20 epochs instead of the 40,000-image subset (same baseline architecture) raises BLEU on the same 500 test images from 79.4 to 86.7 and ExpRate from 0.316 to 0.478 (ExpRate ≤1: 0.680, ≤2: 0.794); the full human training split contains 229,864 expressions.

Catastrophic forgetting. Fine-tuning on photographs degrades handwriting accuracy. On a 200-image slice of the MathWriting test set, the stage-3 (HME100K) checkpoint reaches an ExpRate of only 0.065, whereas a model trained for 20 epochs on the full MathWriting human split reaches 0.445 on the same slice.

B. RQ1: Comparison with pix2tex

Table III compares the two models on identical test images, scored with the same tokenizer and metric code. pix2tex is used off the shelf, without fine-tuning; images on which it raises an error are counted as wrong (empty prediction).

On the 500 MathWriting test images, off-the-shelf pix2tex recognizes none of the expressions exactly (BLEU 0.7), against ExpRate 0.316 for our 40k-image model and 0.478 for our full-data model. Its outputs are typically long, unrelated LaTeX (e.g. nested `array` environments) rather than near misses, so ExpRate ≤2 is also zero. To rule out a broken installation, we rendered two of the same reference expressions as printed formulas: pix2tex transcribed $\frac{q}{A} = h_o(T_o - T_s)$ correctly and failed only on the expression with a cube root. The comparison therefore shows the domain gap of a printed-formula model on handwriting, not the ceiling of transformer architectures: pix2tex was trained mainly on rendered formulas and was not fine-tuned here. On printed formulas (the first 500 im2latex-100k test images) pix2tex is close to our model in BLEU (87.3 vs. 91.5) but lower in exact match (ExpRate 0.352 vs. 0.470). Since pix2tex was trained on rendered formulas, possibly overlapping with im2latex-100k, this row is the most favourable setting for it. On photographed handwriting (HME100K) it fails almost completely: BLEU 4.0, no exact matches, and only 6 of 2,000 expressions within two edits.

TABLE IV

ABLATION ON THE FIRST 500 MATHWRITING TEST IMAGES (SAME SUBSET FOR EVERY ROW). EACH ROW IS FINE-TUNED FROM THE STAGE-1 CHECKPOINT FOR 10 EPOCHS ON THE 40,000-IMAGE SUBSET; ROWS ARE CUMULATIVE.

Variant	BLEU	ExpRate	≤1	≤2
Baseline (stride 32, no coverage)	77.0	0.256	0.472	0.616
+ fine grid (stride 16)	76.7	**0.280**	0.470	0.582
+ coverage attention	76.5	0.256	0.444	0.574
+ photo-style augmentation	**78.5**	0.270	**0.486**	**0.628**

The HME100K test slice used for pix2tex was re-built from the public label file (99,109 labelled images) with the same seed-0 shuffle as our stage-3 split, but the original stage-3 split was carved from roughly 74k images, so the two 2,000-image slices are not guaranteed to coincide. The pix2tex number is unaffected (pix2tex never saw HME100K), but our 88.3 BLEU and the pix2tex row are not on provably identical images. Re-evaluating our stage-3 model on the re-built slice gives BLEU 92.0 and ExpRate 0.618 (≤1: 0.829, ≤2: 0.915), clearly above the 88.3 / 0.524 measured on the original slice. We attribute the gap to probable overlap between the re-built slice and our stage-3 training data, and therefore report the original 88.3 / 0.524 in Tables II and III; the re-built-slice number should be read as an upper bound.

C. RQ3: Ablation

The ablation baseline (BLEU 77.0, ExpRate 0.256) is below the 79.4 / 0.316 of Table II because it is trained for 10 rather than up to 25 epochs. The finer grid gives the highest exact-match rate (0.280, +2.4 points), coverage attention gives no further gain at this budget, and photo-style augmentation gives the highest BLEU (78.5) and near-miss rates (≤1: 0.486, ≤2: 0.628). None of these differences is conclusive: with $n = 500$ the standard error of an ExpRate near 0.26 is about 0.02, so a 95% interval spans roughly ±4 points, wider than any gap in the table, and each row is a single seed. Confidence separation is similar in all four variants (mean confidence 0.94 on correct versus 0.85–0.86 on wrong outputs). We therefore read the ablation as showing that none of the three changes is harmful, not that any one of them is a clear improvement; longer training, several seeds and the full 7,644-image test split would be needed to separate them.

D. Error analysis

Table V breaks ExpRate down by reference length and by the structures an expression contains, using the per-sample predictions on the 500 MathWriting test images. For the 40k-image model, accuracy falls steadily with length, from 0.412 on expressions of at most 10 tokens to 0.128 on those longer than 30; this is the expected weakness of a recurrent decoder, where a single early mistake makes the whole expression wrong. With the full training split, every bucket improves, and the longest bucket rises to 0.487, although it contains only 39 expressions. Expressions with sub- or superscripts are the most common (372 of 500) and among the hardest for both models.

TABLE V

EXPRATE BY EXPRESSION LENGTH AND STRUCTURE ON 500 MATHWRITING TEST IMAGES (NUMBER OF EXPRESSIONS IN PARENTHESES).

Subset	MW 40k	MW full
1–10 tokens (85)	0.412	0.576
11–20 tokens (257)	0.327	0.490
21–30 tokens (119)	0.286	0.378
>30 tokens (39)	0.128	0.487
has `\frac` (141)	0.333	0.496
has sub/superscript (372)	0.298	0.446
has `\sqrt` (39)	0.410	0.641
has `\sum`, `\int`, `\prod` or `\lim` (25)	0.360	0.440
has `\begin` (array/matrix) (15)	0.267	0.600
none of the above (74)	0.324	0.527

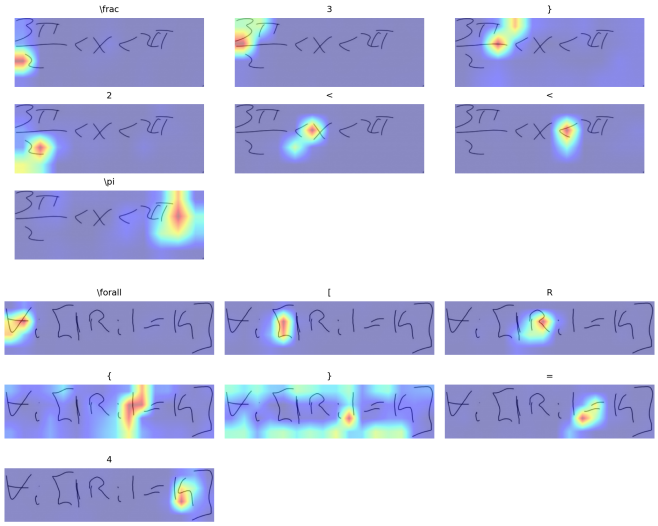


Fig. 1. Attention maps of the full-data model for selected decoding steps (panel title = emitted token). Top: correct, $\frac{3\pi}{2} < x < 2\pi$. Bottom: incorrect, $\forall i\,[\,|R_i| = K\,]$ read with K as "14".

The remaining errors are mostly single-symbol confusions between similar-looking handwritten glyphs rather than structural failures. For example, the full-data model reads $\overline{X} + X_{\xi}$ as $\overline{x} + x_{\xi}$ (case), and $\forall i\,[\,|R_i| = K\,]$ as $\ldots = 14$ (a handwritten K read as the digits 1 and 4).

Fig. 1 shows where the full-data model attends while emitting selected tokens (greedy decoding, so the decoded sequence can differ from beam search; here both agree). Attention moves left to right and settles on the symbol being written. Closing braces of a subscript attend diffusely, since they have no ink of their own. In the incorrect example, attention for the final tokens lands on the handwritten K, which the model splits into the digits 1 and 4.

Confidence. Because the model outputs a length-normalized probability, uncertain predictions can be flagged to the user. Table VI reports ExpRate when only the most confident fraction of predictions is kept. Accuracy rises steadily as coverage falls, so the confidence score ranks predictions usefully, even though mean confidence on wrong answers

TABLE VI
SELECTIVE EXPRATE ON 500 MATHWRITING TEST IMAGES: ACCURACY ON THE MOST CONFIDENT FRACTION OF PREDICTIONS.

Model	100%	80%	60%	40%	20%
MathWriting 40k	0.316	0.388	0.453	0.555	0.700
MathWriting full	0.478	0.568	0.680	0.785	0.870

(0.880 for the 40k model, 0.927 for the full-data model) is only moderately below that on correct answers (0.953 and 0.976).

E. Limitations

- Results in Table II are on 500-image subsets, not full test splits.
- The HME100K test set is a slice we carved from the training labels, not the official test set, so it is in-distribution and not comparable with published results.
- pix2tex is evaluated without fine-tuning on handwriting; a transformer trained under our exact data and budget was not part of this study.
- The checkpoint is selected by validation cross-entropy, not by ExpRate.
- We do not evaluate on the CROHME benchmarks, the most widely reported handwritten-expression test sets.
- Sequential fine-tuning causes forgetting of earlier domains (Section V).

VI. CONCLUSION

A classic ResNet-18 + attention + LSTM model, trained in stages from printed to handwritten to photographed input with a single shared vocabulary, reaches BLEU 79.4 and ExpRate 0.32 on held-out handwritten expressions. Training on the full MathWriting human split raises this to BLEU 86.7 and ExpRate 0.478, while off-the-shelf pix2tex, a printed-formula transformer, recognizes none of the same handwritten expressions exactly. In a short ablation, a finer encoder grid, coverage attention and photo-style augmentation each changed results by less than the sampling error of a 500-image test subset. Future work includes evaluation on full and official test splits including CROHME, joint (rather than sequential) multi-domain training to avoid forgetting, and a preprocessing front end (cropping and binarization) for raw phone photographs.

ACKNOWLEDGMENT

The authors thank the Manipal Institute of Technology for its support of this work, and Kaggle for the GPU compute used for training and evaluation.

REFERENCES

[1] Y. Deng, A. Kanervisto, J. Ling, and A. M. Rush, "Image-to-markup generation with coarse-to-fine attention," in *Proc. ICML*, PMLR vol. 70, 2017, pp. 980–989.

[2] J. Zhang, J. Du, S. Zhang, D. Liu, Y. Hu, J. Hu, S. Wei, and L. Dai, "Watch, attend and parse: An end-to-end neural network based approach to handwritten mathematical expression recognition," *Pattern Recognition*, vol. 71, pp. 196–206, 2017.

[3] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. ICLR*, 2015.

[4] K. Xu *et al.*, "Show, attend and tell: Neural image caption generation with visual attention," in *Proc. ICML*, PMLR vol. 37, 2015, pp. 2048–2057.

[5] Z. Tu, Z. Lu, Y. Liu, X. Liu, and H. Li, "Modeling coverage for neural machine translation," in *Proc. ACL*, 2016, pp. 76–85.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, 2016, pp. 770–778.

[7] L. Blecher, "pix2tex: Using a ViT to convert images of equations into LaTeX code," GitHub repository. [Online]. Available: https://github.com/lukas-blecher/LaTeX-OCR

[8] P. Gervais, A. Fadeeva, and A. Maksai, "MathWriting: A dataset for handwritten mathematical expression recognition," arXiv:2404.10690, 2024.

[9] Y. Yuan *et al.*, "Syntax-aware network for handwritten mathematical expression recognition," in *Proc. CVPR*, 2022, pp. 4553–4562.

[10] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a method for automatic evaluation of machine translation," in *Proc. ACL*, 2002, pp. 311–318.